

Instructions

- Homework 4 was due Monday, August 17th at 18:00 Chicago Time.
 - All numbers and figures come from `homework_4_solution.ipynb`, run from top to bottom.
-

Problem 1: New Data in Our Curve

We'll start by fitting a curve made up of pieces (piecewise linear). Where these pieces intersect are our knots. With K knots the model is:

$$f(x) = w_0 + w_1x + \sum_{j=1}^K w_{1+j}(x - \kappa_j)_+, \quad (u)_+ = \max(u, 0)$$

Each column $(x - \kappa_j)_+$ is zero before the knot κ_j . After the knot it has a slope of 1, so that w_{1+j} is the change in slope at that knot. The model has $p = 2 + K$ parameters.

```

1 import numpy as np
2
3 def basis(x, knots):
4     """Build the basis matrix. One extra column per knot."""
5     cols = [np.ones_like(x), x] + [np.clip(x - k, 0.0, None) for k in knots]
6     return np.column_stack(cols)
7
8 def knots(K):
9     """K knots, evenly spaced inside (0, 1)."""
10    return np.linspace(0, 1, K + 2)[1:-1]
11
12 def fit_ols(A, y):
13     beta, *_ = np.linalg.lstsq(A, y, rcond=None)
14     return beta

```

The x values are the same for all draws; note that only the noise is random (same as Homework 2).

```

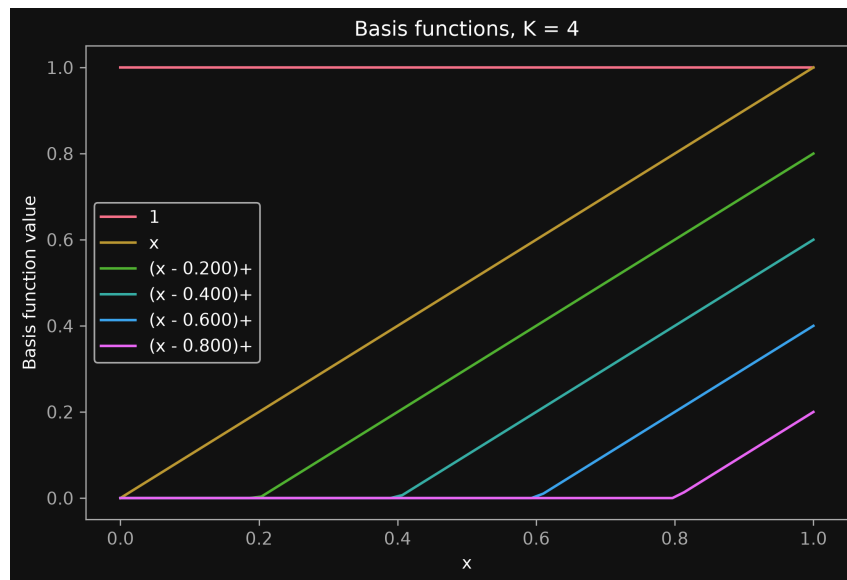
1 N_TRAIN, N_TEST = 60, 500
2 SIGMA = 0.3
3 K_MAX = 18
4 N_REPS = 600
5
6 x_train = np.linspace(0, 1, N_TRAIN)
7 x_test = np.linspace(0.005, 0.995, N_TEST)
8
9 def f_true(x):
10    return np.sin(2 * np.pi * x)
11
12 # in repetition s, use rng = np.random.RandomState(s) and draw
13 # y_train = f_true(x_train) + rng.normal(0, SIGMA, N_TRAIN)
14 # y_test = f_true(x_test) + rng.normal(0, SIGMA, N_TEST)

```

Problem 1.1: Basis Functions

- Plot the columns of `basis(x_train, knots(4))` against x .
- In one sentence, what does the coefficient on the column $(x - \kappa_j)_+$ tell you?

Answer:



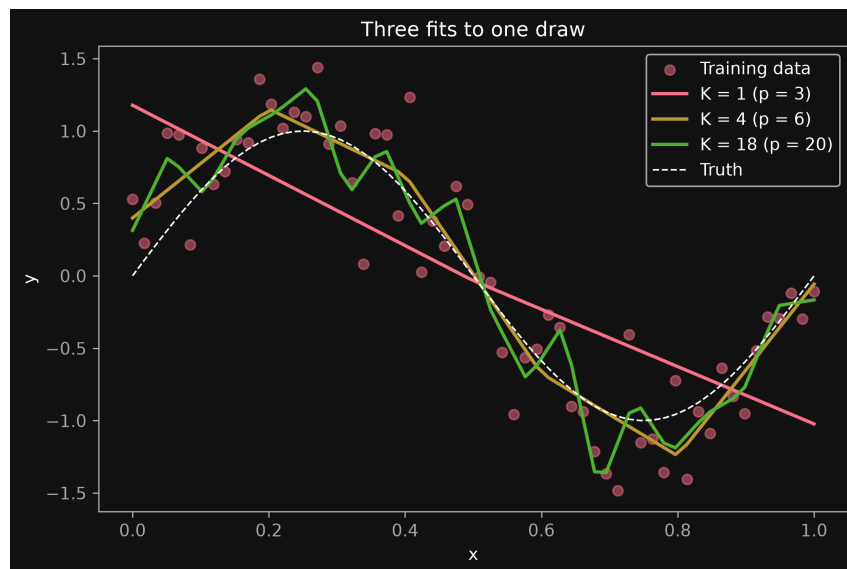
The coefficient on $(x - \kappa_j)_+$ is the change in slope at the knot κ_j .

Problem 1.2: One Set of Data

Use `np.random.RandomState(0)` for the noise. Fit the model for $K = 1$, $K = 4$ and $K = 18$.

- Plot the training data and the three curves on one plot.
- Just from eyeballing; which do you think is the best from balancing bias and variance?

Answer:



$K = 4$. $K = 1$ can bend only once, so it is almost a straight line and misses the shape; $K = 18$ has a knot every three data points and follows single points.

Problem 1.3: Sim

For each $K = 0, 1, \dots, 30$ and each repetition $s = 0, \dots, 999$:

1. Draw new y_{train} and y_{test} with `np.random.RandomState(s)`.
2. Fit the model on the training data only.
3. Record the MSE on the training data and the MSE on the test data.

Take the mean of each over the 600 repetitions.

- Plot the two mean curves against K on one plot (ie. mean MSE and test MSE for each K).
- Give a table of the two values for $K \in \{0, 2, 3, 5, 10, 15, 30\}$.

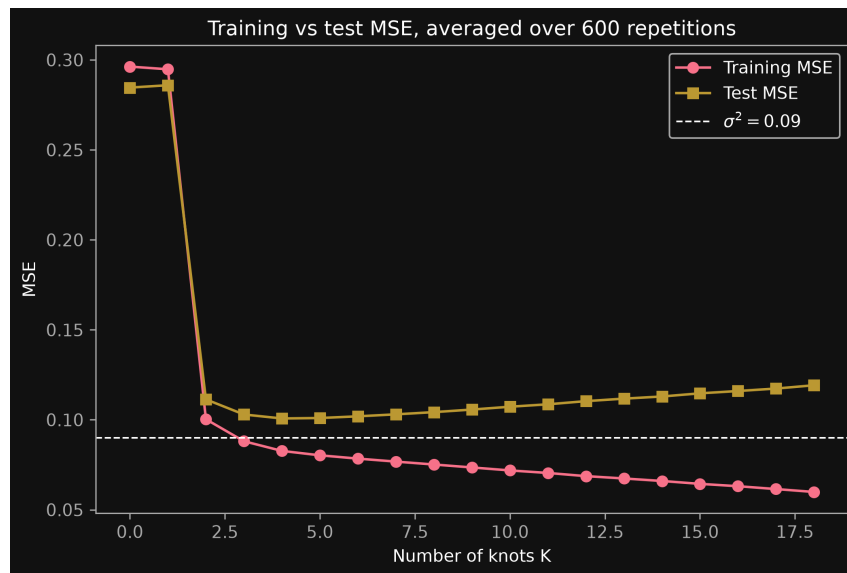
Answer:

The code block above sets $K_{\text{MAX}} = 18$ and $N_{\text{REPS}} = 600$, so the table below runs to $K = 18$ in place of $K = 30$, over 600 repetitions. $K = 4$ is included because it is the minimum.

```

1 A_train = {K: basis(x_train, knots(K)) for K in range(K_MAX + 1)}
2 A_test = {K: basis(x_test, knots(K)) for K in range(K_MAX + 1)}
3
4 ins_all, oos_all = [], []
5 for s in range(N_REPS):
6     rng = np.random.RandomState(s)
7     y_tr = f_true(x_train) + rng.normal(0, SIGMA, N_TRAIN)
8     y_te = f_true(x_test) + rng.normal(0, SIGMA, N_TEST)
9     ins_row, oos_row = [], []
10    for K in range(K_MAX + 1):
11        beta = fit_ols(A_train[K], y_tr)
12        ins_row.append(np.mean((A_train[K] @ beta - y_tr) ** 2))
13        oos_row.append(np.mean((A_test[K] @ beta - y_te) ** 2))
14    ins_all.append(ins_row)
15    oos_all.append(oos_row)
16
17 in_sample = np.mean(ins_all, axis=0)
18 out_sample = np.mean(oos_all, axis=0)

```



K	p	training MSE	test MSE
0	2	0.2963	0.2846
2	4	0.1003	0.1113
3	5	0.0881	0.1029
4	6	0.0827	0.1007
5	7	0.0802	0.1010
10	12	0.0718	0.1073
15	17	0.0644	0.1147
18	20	0.0599	0.1192

Problem 1.4: What the Plot Shows

- Does the training MSE ever increase when you add a knot?
- In one sentence, why is that?
- Give the value of K with the lowest test MSE.

Answer:

- No. The training MSE falls at every step, from 0.2963 to 0.0599.
- Each extra knot adds a parameter, and more parameters let the fit pass closer to the training points. Adding a column can never raise the minimized sum of squares, because the old solution is still available with a zero coefficient on the new column.
- The test MSE is lowest at $K = 4$ ($p = 6$), at 0.1007. It then rises at every step, to 0.1192 at $K = 18$.

Problem 1.5: Theory

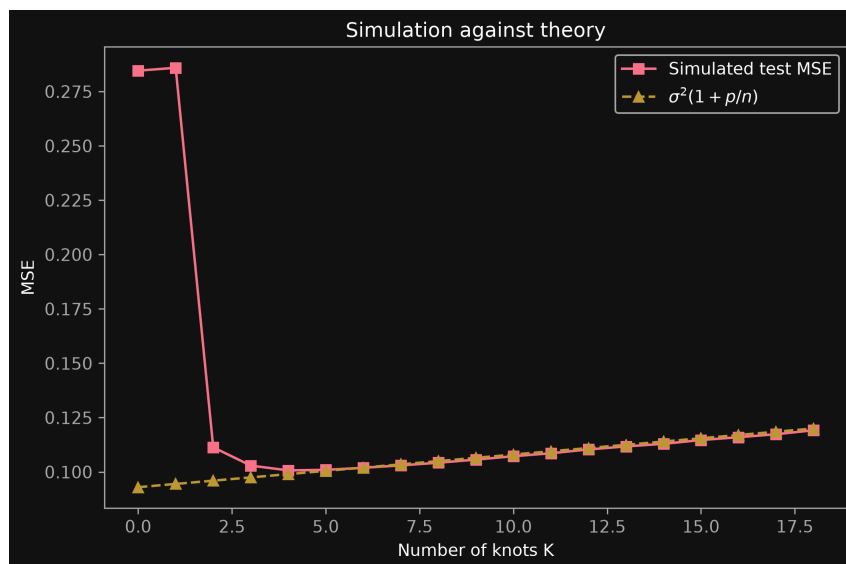
For a model with p parameters that has the correct shape, the expected test MSE is roughly:

$$\sigma^2 \left(1 + \frac{p}{n}\right)$$

Here $p = 2 + K$ and $n = 60$.

- Draw this theoretical curve on top of your test MSE curve (from the previous question).
- Give the range of K where $|\text{simulated}/\text{formula} - 1| \leq 0.03$.
- In 1–2 sentences, what does the formula leave out at small K ?

Answer:



K	simulated	$\sigma^2(1 + p/n)$	gap
0	0.2846	0.0930	+206.0%
2	0.1113	0.0960	+16.0%
3	0.1029	0.0975	+5.6%
4	0.1007	0.0990	+1.7%
5	0.1010	0.1005	+0.5%
10	0.1073	0.1080	−0.7%
15	0.1147	0.1155	−0.7%
18	0.1192	0.1200	−0.7%

The simulation is within 3% of the formula for $K = 4$ to $K = 18$, the whole range available. ($K = 3$ is 5.6% away, outside; $K = 4$ is 1.7% away, inside.)

The formula charges only for estimating p parameters, and it assumes the model can reproduce the true shape. At small K it cannot: a line with one or two bends is nowhere near $\sin(2\pi x)$, so the gap at $K = 0$ and $K = 2$ is squared bias, which the formula leaves out entirely.

Problem 2: Which Knots Are Real?

In the previous question, the data was a sin wave. This time, the data really is piecewise linear; the slope changes at $x = 0.3$ and at $x = 0.7$. There are 19 possible knots, and we need to see if we can find the real ones.

```

1 TRUE_KNOTS = [0.3, 0.7]
2 TRUE_BETA = np.array([0.0, 2.0, -8.0, 10.0]) # intercept, slope, then the two
      slope changes
3 CANDIDATE_KNOTS = np.linspace(0.05, 0.95, 19)
4 N2, SIGMA2 = 200, 0.3
5
6 rng = np.random.RandomState(7)
7 x2 = rng.uniform(0, 1, N2)
8 y2 = basis(x2, TRUE_KNOTS) @ TRUE_BETA + rng.normal(0, SIGMA2, N2)
9
10 # scikit-learn adds its own intercept, so remove the first column
11 A_full = basis(x2, CANDIDATE_KNOTS)
12 design = A_full[:, 1:]

```

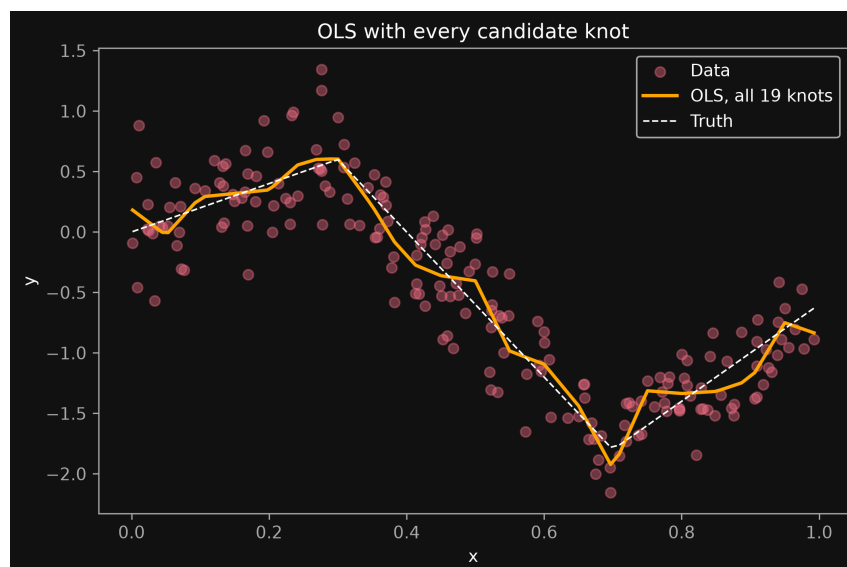
Problem 2.1: Plain OLS

Fit OLS with all 19 possible knots.

- Give the largest coefficient of the 19 knot columns, in absolute value.
- Plot the fitted curve.
- Can you find the two real knots from the coefficients? Give a why or why not reason in one sentence.

Answer:

The largest knot coefficient is **23.28** in absolute value. The largest true slope change is 10, so the ratio is **2.33**.



No. The coefficients alternate in sign again and again ($+10.55, -5.72, +0.02, +4.35, -4.74, -8.03, \dots$), so nothing marks out 0.30 and 0.70. Knots that sit close together give almost the same column, and OLS is free to place a large positive coefficient on one and cancel it with a large negative one on its neighbor without changing the fitted values much.

Problem 2.2: Lasso to the Rescue?

Fit `LassoCV(cv=5, random_state=0, max_iter=200000)` on `design`.

- Give the α/λ that it selects.
- Give the number of knots with a coefficient that is not zero (or extremely close to it). Are both real knots in the list?

Answer:

- $\alpha = 0.00054$.
- 4 of 19 knots survive: $\{0.05, 0.25, 0.30, 0.70\}$.
- Yes, both real knots are in the list, along with two false ones. Note that 0.25 sits next to the real knot at 0.30; when two columns are nearly identical, Lasso often splits one real effect between them.

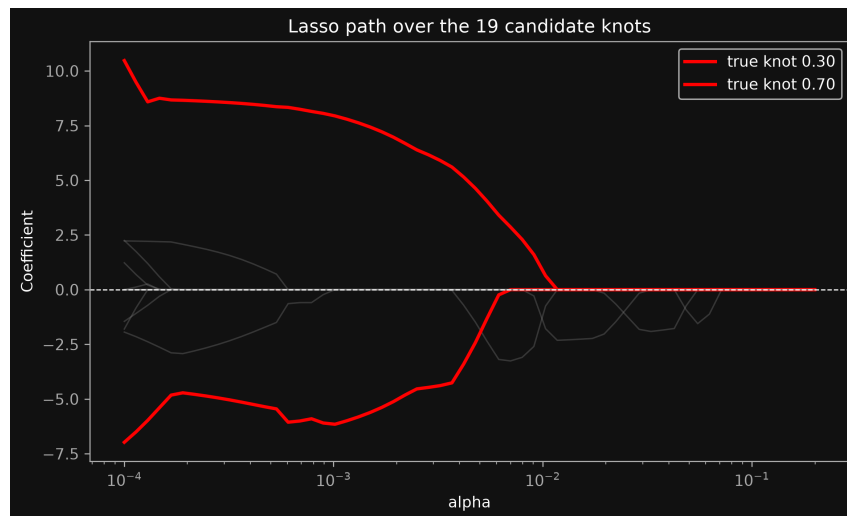
Problem 2.3: Change the Penalty

Fit `Lasso(alpha=a, max_iter=200000)` for $a \in \{0.001, 0.005, 0.01, 0.02, 0.05, 0.1\}$.

- Give a table with one row for each α : the number of knots that stay, and their values.
- Is there an α that keeps exactly $\{0.3, 0.7\}$ and nothing else?

Answer:

α	knots that stay	values
0.001	2	$\{0.30, 0.70\}$
0.005	3	$\{0.25, 0.30, 0.70\}$
0.010	3	$\{0.20, 0.25, 0.70\}$
0.020	2	$\{0.15, 0.20\}$
0.050	2	$\{0.10, 0.15\}$
0.100	0	$\{\}$



Yes: $\alpha = \mathbf{0.001}$ keeps exactly $\{0.30, 0.70\}$ and nothing else. A larger penalty does not keep helping – by $\alpha = 0.02$ the two survivors are $\{0.15, 0.20\}$ and both are wrong.

Problem 2.4: Why the Two Disagree

In a few sentences: why do we tend to keep more knots than are really needed?

Answer:

Cross-validation picks α to make the test error small, not to recover the true knots. Those are different targets. One or two extra small coefficients cost almost nothing in test error, and they insure against dropping a real knot, which is expensive. So the α that is best for prediction sits below the α that is best for selection – here 0.00054 against 0.001 – and cross-validation ends up keeping too many knots.

Problem 3: Your Own Basis, and a Rule That OLS Cannot Follow

Like we saw in class, Gaussian blobs evenly spaced by years can be hard to fit a yield curve to. The reason is that most of the "interesting" part of the yield curve happens in the first few years to maturity.

A lognormal basis function is a Gaussian blob in log tenor:

$$\phi_i(\tau) = \exp\left(-\frac{(\log \tau - \mu_i)^2}{2s^2}\right)$$

```

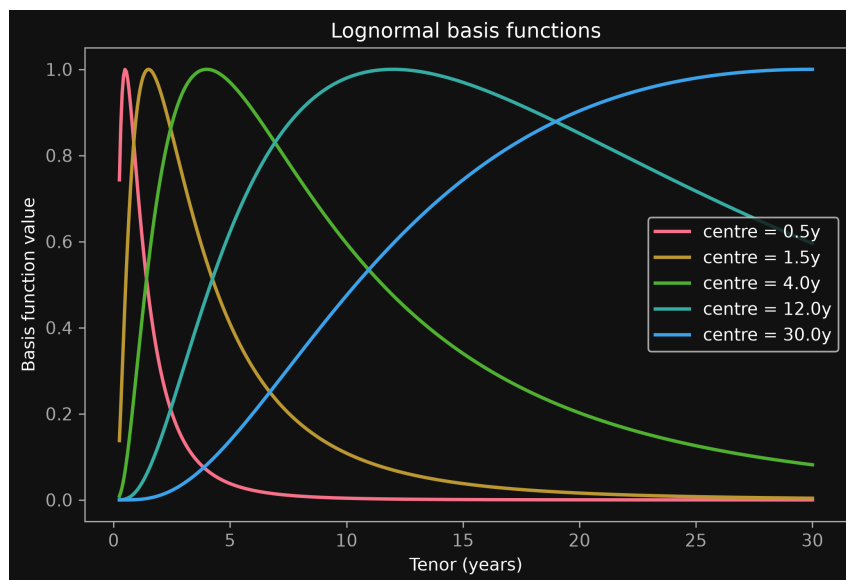
1 def lognormal_basis(tau, mu, s):
2     return np.exp(-((np.log(tau) - mu) ** 2) / (2 * s ** 2))
3
4 TENORS = np.array([0.25, 0.5, 0.75, 1, 2, 3, 4, 5, 7, 10, 12, 15, 20, 25, 30],
5                    dtype=float)
6 CENTRES = np.log(np.array([0.5, 1.5, 4.0, 12.0, 30.0]))
7 WIDTH = 0.9
8
9 def true_curve(tau):
10     return 0.02 + 0.01 * np.sqrt(tau)
11
12 rng = np.random.RandomState(0)
13 y3 = true_curve(TENORS) + rng.normal(0, 0.0004, len(TENORS))
14
15 tau_fine = np.linspace(0.25, 30, 500)
16
17 def ln_design(tau):
18     return np.column_stack([lognormal_basis(tau, m, WIDTH) for m in CENTRES])
19
20 A3 = ln_design(TENORS)

```

Problem 3.1: The Basis

- Plot the five basis functions against τ on `tau_fine`.
- Take the blob at 0.5 years and the blob at 30 years. For each one, give (roughly) the width in years over which the blob is at or above 0.5.

Answer:



$\phi_i(\tau) \geq 0.5$ when $(\log \tau - \mu_i)^2 \leq 2s^2 \log 2$, so the half-max range is $\tau \in [e^{\mu_i - s\sqrt{2\log 2}}, e^{\mu_i + s\sqrt{2\log 2}}]$, a fixed ratio of $e^{\pm 1.06}$ around the centre. Measured on `tau_fine`, which stops at 0.25 and 30:

centre	on the grid	width	exact	width
0.5y	0.25 – 1.44	1.19	0.17 – 1.44	1.27
1.5y	0.55 – 4.30	3.76	0.52 – 4.33	3.81
4y	1.44 – 11.52	10.08	1.39 – 11.54	10.16
12y	4.18 – 30.00	25.82	4.16 – 34.62	30.47
30y	10.44 – 30.00	19.56	10.40 – 86.56	76.17

The blob at 0.5 years is about 1.2 years wide. The blob at 30 years is at least 19.6 years wide on the grid, and 76 years wide if you do not stop at the 30-year point. Either way it is the blob at the front that is narrow, which is the point of working in $\log \tau$: the basis is fine where the curve moves and coarse where it does not.

Problem 3.2: Ordinary Least Squares

Fit the five weights with OLS on `A3` and `y3`. Store them as `w_ols3`.

- Give the five weights, and the number of them that are negative.
- In one sentence, why is a negative weight hard to explain when each basis function is a blob of yield?

Answer:

centre	0.5y	1.5y	4y	12y	30y
$\hat{w}$	0.03083	-0.00688	0.04134	-0.00927	0.07430

2 of the 5 weights are negative. The smallest is -0.00927 .

Each basis function is a blob of yield sitting at one tenor, so a negative weight says the blob at 1.5 years and the blob at 12 years *remove* yield. Nothing in the market does that, and once a weight can go negative it stops answering the question the basis was built to answer – how much curve sits at each tenor.

Problem 3.3: Non-Negative Weights

In class we said that we could stop the negative weights with the rule $w \geq 0$. Apply that rule:

```
1 from scipy.optimize import minimize, nnls
2
3 res = minimize(lambda w: np.sum((A3 @ w - y3) ** 2), np.maximum(w_ols3, 0.0),
4                 method="L-BFGS-B", bounds=[(0.0, None)] * A3.shape[1],
5                 options={"maxiter": 100000, "ftol": 1e-18, "gtol": 1e-14})
```

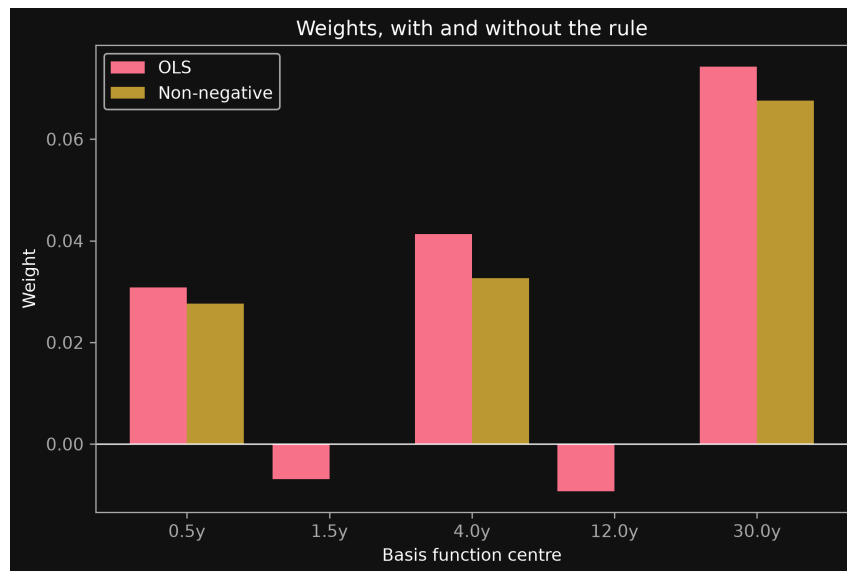
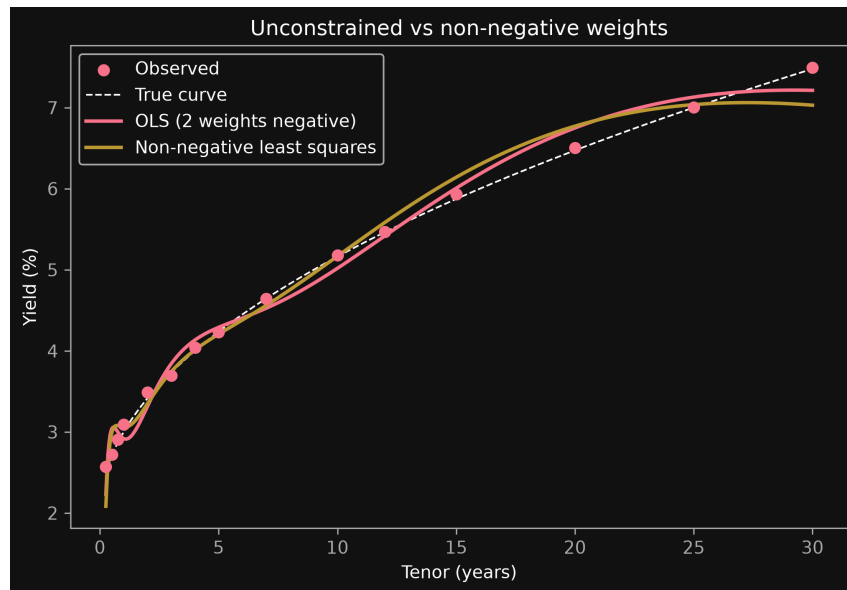
- Give the five new weights.
- Give the RMSE against `true_curve(tau_fine)`.
- Plot your two fitted yield curves against the data, which do you think looks better?

Answer:

centre	0.5y	1.5y	4y	12y	30y
$\hat{w}$	0.02765	0.00000	0.03268	0.00000	0.06763

The two weights that were negative are now exactly zero. The rule does not shrink them, it removes those two basis functions from the model. `scipy.optimize.nnls` returns the same answer to 4.05×10^{-9} , so the bounded solver found the true optimum.

RMSE against `true_curve(tau_fine)` is **19.87** bp, against 16.70 bp for unconstrained OLS, and the SSE against `y3` is 1.373 times the OLS value.



The constraint costs accuracy on both measures. On the SSE against y_3 it has to: OLS already minimizes that, and the constrained fit is a restriction of the same problem. On the RMSE against the true curve it does not have to – a constraint that happens to point at the truth can help – but here it does not. The two curves are hard to tell apart by eye, and both track the data. The non-negative one is the better answer anyway, because the five weights it returns can be read as an allocation across tenors and the OLS weights cannot. If all you want is a curve, take OLS; if you want the weights to mean something, pay the 3 bp.

Problem 4: Switching Up Losses

Sometimes, data might have bad values.

```

1 x4 = np.linspace(0, 1, 120)
2 clean_curve = np.sin(2 * np.pi * x4)
3
4 rng = np.random.RandomState(5)
5 y4 = clean_curve + rng.normal(0, 0.05, 120)
6 bad_idx = rng.choice(120, 8, replace=False)
7 y4[bad_idx] += rng.choice([-1, 1], 8) * 1.5      # 8 bad values
8
9 B4 = basis(x4, knots(8))                        # 10 parameters
10 w_ols = fit_ols(B4, y4)                        # the start point for both fits below

```

Use these settings for every fit in this problem. Start each fit from the OLS answer.

```

1 from scipy.optimize import minimize
2
3 OPT = {"maxiter": 50000, "maxfun": 200000, "ftol": 1e-15, "gtol": 1e-12}
4 # minimize(loss, w_ols, method="L-BFGS-B", options=OPT)

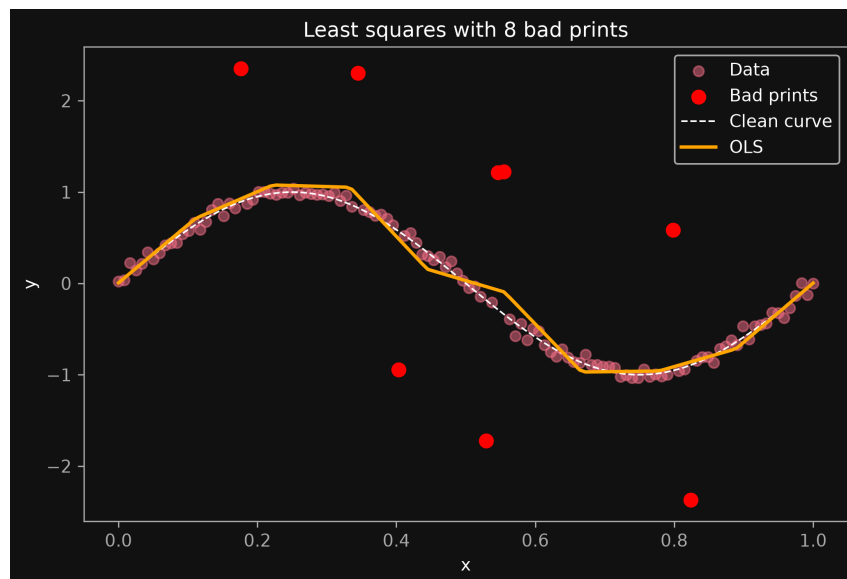
```

Problem 4.1: Least Squares

Plot the data, the OLS curve, and `clean_curve` on one set of axes. Show the 8 bad values in a different color.

Answer:

The 8 bad values are at index 21, 41, 48, 63, 65, 66, 95, 98.



The curve moves toward every bad value.

Problem 4.2: Two Other Losses

Write these two losses yourself, then run them through `scipy.optimize.minimize`:

- the Huber loss with $\delta = 0.1$:

$$\rho_{\delta}(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta (|r| - \frac{1}{2}\delta) & |r| > \delta \end{cases}$$

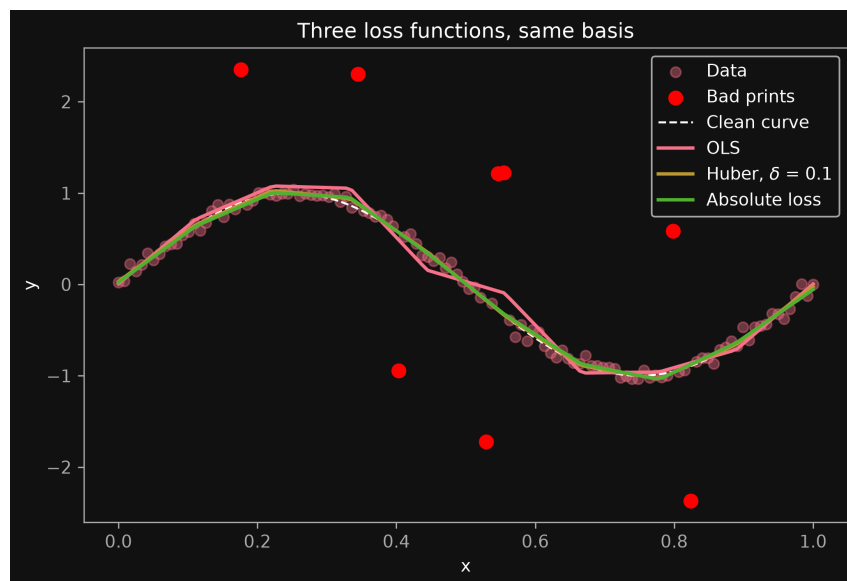
- the (almost) absolute loss. Use the smooth version below (so that it's differentiable at $r = 0$).

$$\rho(r) = \sqrt{r^2 + \varepsilon^2}, \quad \varepsilon = 10^{-3}$$

Plot these two curves and the OLS curve from 4.1 together with `clean_curve`.

Answer:

```
1 def huber_loss(r, delta):
2     a = np.abs(r)
3     return np.where(a <= delta, 0.5 * r**2, delta * (a - 0.5 * delta))
4
5 w_huber = minimize(lambda w: np.sum(huber_loss(B4 @ w - y4, 0.1)), w_ols,
6                    method="L-BFGS-B", options=OPT).x
7
8 # |r| has a sharp corner at 0 that a solver cannot follow, so smooth it slightly
9 w_abs = minimize(lambda w: np.sum(np.sqrt((B4 @ w - y4) ** 2 + EPS**2))), w_ols,
10                  method="L-BFGS-B", options=OPT).x
```



The smoothing matters. With the sharp corner left in place, a gradient-based solver stops early and returns a worse answer than the true minimum of $\sum |r|$.

Problem 4.3: Which Loss Wins

- Give the RMSE of the three fits against `clean_curve`.
- Which one is lowest, and why?
- Now give the RMSE of the three fits against `y4` as well. Which is better, and why do you think that is?

Answer:

loss	RMSE vs clean curve	RMSE vs y4
OLS	0.0860	0.3844
Huber ($\delta = 0.1$)	0.0190	0.3926
absolute	0.0257	0.3927

Huber wins on the clean curve, at 0.0190 against 0.0257 for the absolute loss and 0.0860 for OLS.

Squared error grows with r^2 , so a distant point dominates the objective and the curve must move toward it. Compare the share of the objective that a residual of 1.5 takes against one of 0.1: OLS gives it $225\times$ the weight, Huber $29\times$, the absolute loss only $15\times$.

Huber wins because it does both jobs at once. It is quadratic near zero, so it uses the 112 good points as efficiently as OLS does, and linear far out, so the 8 bad values get a bounded vote. The absolute loss is linear everywhere: it handles the bad values but throws away the same information on the good points, so it lands between OLS and Huber.

OLS wins on y4 and loses on the clean curve. That ordering is guaranteed, not luck: OLS minimizes the squared error against `y4` by construction, so nothing can beat it there – and `y4` is the series that holds the 8 bad values. Fitting the data you were given and fitting the process that generated it are not the same objective.

Problem 4.4: The Effect of δ

Fit the Huber loss again for $\delta \in \{0.01, 0.05, 0.1, 0.5, 1.0, 5.0\}$.

- Give the RMSE against `clean_curve` for each δ .
- What is Huber loss like for very small δ ? And what is it like for really big δ ?

Answer:

δ	RMSE vs clean curve	largest $ r $
0.01	0.0228	1.5536
0.05	0.0201	1.5525
0.10	0.0190	1.5571
0.50	0.0365	1.6026
1.00	0.0710	1.6806
5.00	0.0860	1.6895

As $\delta \rightarrow 0$ Huber becomes the **absolute loss**; as $\delta \rightarrow \infty$ it becomes **OLS**.

Both ends show up in the table. At $\delta = 5.0$ the RMSE is 0.0860, which is OLS exactly: no fit here leaves a residual larger than 1.6895, well below 5, so no point ever reaches the linear part and Huber is minimizing $\frac{1}{2}r^2$ everywhere. At $\delta = 0.01$ the RMSE is 0.0228, close to the 0.0257 of the absolute loss.

The table has an interior minimum, at $\delta = 0.1$. Too large and Huber behaves like OLS and chases the bad values; too small and it behaves like the absolute loss and gives up efficiency on the good points.